

Quorum — handoff for an AI agent

Everything an agent needs to pick this project up cold. Written 25 September 2026 against commit `4ffc1f7` on branch `claude/nice-ramanujan-jt9qjv` of `github.com/aabhasamol/nobody-fronts` (pull request #1). Read this, then `README.md`, then `app/engine.py`. The other documents are indexed in §13.

1. What this is, in one page

Quorum is a trip-planning agent for a WhatsApp group of friends or families. Its one rule: **the organiser never fronts money and never chases anyone**. It exists as an entry to *The Ken's Great Rewiring*, Round 2, opening 07 of 16 ("Planning the trip"), by Aabhas, Aditi and Sayan (IIM Calcutta). The competition asks teams to picture an agent running on three rails — **Pine Labs** (payments), **Gnani** (voice), **Delhivery** (logistics) — and to say what exists, what must be built, and where it breaks. The Round-2 form closes Friday 25 September 2026, 11:59 PM IST; the answers live in `docs/round2-answers.md`.

The product is a **state machine** (`app/engine.py`) plus three **rail interfaces** with mock and real adapters, a FastAPI surface, a demo UI, a terminal demo, and 53 tests. It runs offline on the mocks; the real adapters are written against the rails' documentation and marked `[verify]` wherever a live call has not been made.

The loop, in one sentence: the organiser gives a rough budget and a maximum overshoot → the agent gathers each payer's constraints privately → builds one itinerary inside the budget → each payer votes privately, only the tally reaches the group → the yes-voters set the trip's budget (heads × their lowest per-head ceiling) and the plan is re-sized for exactly them → each payer blocks share × (1 + overshoot) by UPI mandate or payment link → the agent captures each share into Quorum's merchant account (the pool) and pays every supplier from it → after booking it re-books cancelled legs inside each member's headroom and re-routes missed departures.

Personas: bachelors and families. No corporate.

2. Decisions the team made (in order; do not re-litigate)

1. **Two-step, not one:** a private vote first, then money. Approving a mandate is not the vote.
2. **Voice is for P0 things only.** Three reasons exist (§6). Members are never called to chase; listed hotels are never called to check the listing.
3. **Gnani's API is text-to-speech and speech-to-text**, nothing more. The voice rail is built from those two verbs plus a telephony seam that is not Gnani's.
4. **The Gnani speech key is committed** in `app/keys.py` on purpose (private repo, one-month life, one holder, speech-only scope, budget-capped). Revoke it after the competition.
5. **The pool is Quorum's merchant account** (option 1 of the four pooling designs). The organiser is never in the money path. Quorum pays suppliers from the pool. The pool can never go negative: nobody fronts, Quorum included.

6. **The yes-voters set the budget:** the heads they pay for $\times$ the lowest per-head ceiling among them. The proposal before the vote is held to the organiser's rough budget $\times$ (1 + overshoot) per head.
 7. **A member is a payer, not necessarily one traveller.** Gathering asks privately how many people they pay for. Rooms, seats, shares, the budget and the mandates follow the head count; the vote is one per payer.
 8. **Two ways to block, the payer's choice:** a UPI mandate (nothing moves) or a payment link. On the link a card is held (Pine Labs `pre_auth`); anything that cannot hold pays now and waits in the pool, refunded on lapse. The float on prepaid money is Quorum's, gains and losses; it is logged as an estimate and labelled an idea.
 9. **Credit instruments:** card holds with EMI, and pay-later via LazyPay inside its limit. **Quorum never lends and never charges interest:** the lender fronts, Quorum is settled in full at booking.
 10. **Nobody revokes from their own app** (Pine Labs' OTM doc). A member asks Quorum, and Quorum releases.
 11. **The organiser sets the authorisation deadline.**
 12. **Re-sizing for whoever is in is one hook** (`Engine._retarget`). Rooms re-split today; a car or an activity would re-size in the same place. "Two cars for ten become one SUV for seven" was an example, not a feature request.
 13. **Personas shape defaults and are never announced.**
 14. **The place does the convincing:** every message that had to go anyway carries one line about the destination, never repeated to the same person. The outs get one private nudge and 12 hours to flip in.
 15. **Missed departures are the member's:** every way to still get there today (later flights, a train, an Uber Outstation cab) goes to them, no refund, their headroom or a top-up.
 16. **Fewest bottlenecks:** six taps to build the plan; names, dates of birth, food and medical only after a yes.
 17. **The group is fun, the DM is where the decision is made:** a DM carries everything needed to weigh a yes (number, coverage, when money moves, what's refundable, deadline, what silence means) and then leaves the person alone. `member_takes_time` (`POST /time`) suppresses the halfway reminder; the deadline never moves. No guilt lines, no names in the group, never a call to chase.
-

3. The flow, state by state

States: `INITIATED` $\rightarrow$ `GATHERING` $\rightarrow$ `PLANNING` $\rightarrow$ `VOTING` $\rightarrow$ `AUTHORISING` $\rightarrow$ `BOOKING` $\rightarrow$ `BOOKED`, with `LAPSED` (nothing booked, nothing charged) and `CLOSED` (organiser closed it). Every user-visible sentence goes through `Engine._say` into `Trip.events`, which is the transcript the UI and `demo.py` render. Every event carries a priority: **P0** a phone call, **P1** a text now, **P2** a text that can wait.

The group chat gets four kinds of post only: kickoff, the tally (with the budget and re-size), the booking, and disruptions. Everything else is a private DM.

State	What happens	Constants
INITIATED	Organiser adds Quorum: destination, dates, occasion (leisure / wedding / offsite / pilgrimage), rough budget per head, max overshoot, optional venue, deadline hours, persona. Kickoff post carries one general lore line	<code>trigger()</code>
GATHERING	DM per member, six taps: heads they pay for, start city, return city, dates they can't do, per-head ceiling, interests + must-haves. Silence at the deadline = just them, home city both ways, organiser's figure as ceiling, one head	<code>GATHER_WINDOW_H = 24</code>
PLANNING	Trip side: stays ranked by occasion (leisure: price; wedding/offsite/pilgrimage: km to venue). Phone-only stays get a supplier call (retry once on no answer; the call's rate replaces the listing; UNAVAILABLE $\Rightarrow$ never call again, <code>trip.stays_out</code>). Travel side: per payer, out from start city and back to return city, searched separately, seats = party. Legs by occasion: wedding/offsite land before <code>FUNCTION_HOUR = 18</code> ; pilgrimage/family daytime; leisure cheapest. Leisure trips add one pre-bookable activity shared by ≥ 2 payers as an essential, plus on-the-day suggestions. Fit: proposal $\Rightarrow$ every head $\leq$ organiser limit, or a minority over (flagged privately); after a vote $\Rightarrow$ total $\leq$ the yes-voters' budget. No fit $\Rightarrow$ the organiser is told the cheapest plan and what drives it, and waits (<code>awaiting_organiser = "no_fit"</code> ; <code>organiser_adjusts</code>)	<code>HOLD_HOURS = 48</code> on supplier calls
VOTING	One plan DM'd to each payer: their legs (per seat), the stay split, essentials, on-the-day list, one lore line, their all-in per head against their ceiling, their share for their party, what they'd authorise. Yes/no with a reason. A yes triggers the details DM (names, DOB, food; medical and pets for families). One reminder at the halfway mark. Majority of <i>members</i> ($\lfloor n/2 \rfloor + 1$) passes. Fails $\Rightarrow$ up to <code>MAX_REVISIONS = 2</code> revisions built from the reasons: price words $\Rightarrow$ a cheaper stay, else one night fewer; otherwise a different stay. After that the organiser decides (<code>organiser_decides(go)</code>): book for the yeses, or close)	<code>VOTE_WINDOW_H = 24</code>
AUTHORISING	<code>Trip.set_budget(ins)</code> : floor = lowest ceiling among the payers in, total = heads $\times$ floor. <code>_retarget</code> re-sizes rooms/seats and re-adds legs for anyone joining. Over the total $\Rightarrow$ <code>_back_to_group</code> (release all blocks, revise for those in, new vote). Else one block per payer for share $\times$ (1 + overshoot), rounded up to ₹100; DM offers UPI mandate / payment link (card hold, EMI tenures, pay-later if $\leq$ limit, or pay now). The outs get one nudge with their all-in if they join and <code>FLIP_WINDOW_H = 12</code> to <code>member_flips</code> . One reminder at the halfway mark. Deadline $\Rightarrow$ non-blockers drop, the rest re-size and re-check (caps and total); over $\Rightarrow$ back to the group; nobody left $\Rightarrow$ lapse. <code>member_withdraws</code> = ask Quorum to release; treated as a dropout	organiser's <code>auth_window_h</code> (default 48)
BOOKING	Re-hold an expired phone hold. Re-price legs at live fares: inside the cap $\Rightarrow$ absorbed (shown on the receipt); over $\Rightarrow$ only that payer is asked for a top-up (a fresh block), booking waits. Then <code>_capture_all</code> : one debit per payer; any failure $\Rightarrow$ refund everyone already debited, release the rest, LAPSED, group told, organiser can rerun. Then inventory: stays (phone hold converted), activities, legs (seats), each paid from the pool with a logged payout; receipts DM'd; one group post	

State	What happens	Constants
BOOKED	<code>disrupt(leg)</code> : carrier cancels $\Rightarrow$ refund into the pool, alternatives priced, re-book inside headroom + refund (wedding picks an arrival before the function, else cheapest); a re-booked arrival $\geq$ <code>LATE_ARRIVAL_HOUR = 20</code> at a phone-only stay $\Rightarrow$ late-arrival call . Over the headroom $\Rightarrow$ text with two options; unanswered <code>ESCALATE_AFTER_MIN = 20</code> $\Rightarrow$ escalation call ; choice $\Rightarrow$ top-up block $\Rightarrow$ re-book. <code>missed(leg)</code> : no refund; <code>search_alternatives</code> (later flights, first tomorrow, a train and an Uber Outstation cab where the road ≤ 700 km), three options soonest-first, same top-up/escalation path. <code>member_exits</code> after booking: their legs stay theirs, the new stay split goes to the group, humans decide. Daily place fact per traveller from booking to departure (<code>LORE_HOUR = 9</code> , once per date in <code>trip.lore_sent</code> , skipped on countdown days, silent once the table runs dry); countdown DMs with PNR at T-7 and T-1 (<code>DRIPS</code>)	

Amounts. Everything in the engine is INR integers, printed with Indian grouping (`models.inr`: ₹1,00,000). Adapters convert to paisa at the edge. Caps round up to the next ₹100 (`ceil100`).

4. Money

Per head vs share. `Plan.per_head(payer)` is one of that payer's people: their legs per seat + the stay per head (rooms = $\lceil \text{heads}/2 \rceil$, twin sharing, split over all heads) + pre-booked essentials per head. `Plan.share(payer)` = per head $\times$ party. `Plan.total()` = Σ shares. `Plan.stay_total(stay)` = rooms $\times$ rate $\times$ nights is what the supplier is paid; per-head rounding leaves a rupee or two in the pool.

Caps and headroom. Each payer's block is `cap = ceil100(share  $\times$  (1 + overshoot))`. `Authorisation.headroom()` is what can still be debited without a new approval: `amount - captured` for multi-debit instruments, 0 once a single-capture instrument has captured. `Engine._present(payer, amount)` debits the share block first, then the top-up block. `Engine._headroom` sums both.

Instruments (`models.INSTRUMENTS`; the member fixes one when they approve):

Key	What it is	Multi-debit	Prepaid	Life	Max	Notes
UPI_RESERVE	UPI Reserve Pay: block once, debit many times	yes	no	60 d	₹1,00,000	Default. Headroom stays live after the share is taken: re-bookings need no tap
UPI_OTM	UPI one-time mandate	no	no	60 d	₹1,00,000	One capture, partial allowed, merchant releases the rest
CARD_PREAUTH	credit-card hold on a pre-authorised payment link	no	no	7 d	none	One capture at booking; rest of hold released; <code>emi_months</code> optional (issuer collects; Quorum settled in full)

Key	What it is	Multi-debit	Prepaid	Life	Max	Notes
PREPAID	payment link paid with a method that cannot hold	yes (it is cash)	yes	none	none	Money in the pool at once; refunded in full on lapse/withdrawal; spare above the share is cash headroom, returned after the trip; the float is Quorum's (<code>FLOAT_RATE = 0.065</code> , logged as an idea)
BNPL	pay later via LazyPay	no	no	none	₹30,000 [verify]	Lender fronts; Quorum settled in full; member repays LazyPay after the trip

The pool (`PaymentsRail.pool()`): captured – refunded – paid out + supplier refunds. `pay_supplier` must refuse to take it negative. Payouts are logged per booking (`Payout` : supplier, amount, purpose STAY / ACTIVITY / LEG / REBOOK, reference, method). A carrier's refund on a cancelled leg comes back in (`receive_refund`) and the new ticket is paid from it. In the demo the pool closes to ₹0 after booking and holds exactly the members' leftovers afterwards.

Rollback. `_capture_all` loops one capture per payer. On `CaptureFailed` , everyone already debited is refunded (prepaid: the whole payment), the rest released, state LAPSED. This is the wall we describe to Pine Labs: a compensating rollback, not atomicity.

5. Voice: calls are for P0 things

`Engine.P0_CALLS` is the whole list; `Engine._call` is the only way the engine dials and it asserts the reason is one of them:

Reason	When
SUPPLIER_AVAILABILITY	a phone-only stay, before the plan can go to the vote or the booking (rooms, rate, refund terms, 48 h hold). Retry once on no answer
LATE_ARRIVAL	a re-booked arrival at or after 20:00 at a phone-only stay
DISRUPTION_UNANSWERED	a cancelled or missed leg, the options text unanswered for 20 minutes

Everything else is a text: gathering, the plan, the vote, the UPI request, both reminders, receipts, a failed debit, the nudge to the outs, the daily place fact, the countdowns.

The rail (`app/rails/gnani.py`): Gnani's API is TTS (`POST https://api.vachana.ai/api/v1/tts/inference`) and STT (`POST https://api.vachana.ai/stt/v3`), key in `app/keys.py` , ₹27 per audio-hour / per 10k characters, cached TTS, `CallBudget` refuses the 41st billable call per checkout (`.gnani_budget.json` , git-ignored). A call = `Telephony.dial` → play TTS of each scripted line → `listen` → STT → rule-based field reading (`parse_yes_no` with last-verdict-wins and Hindi idioms, `parse_amount` with hazaar/sau, `parse_choice` with explicit "option two" beating a bare "one"). `FileTelephony` (default) writes agent audio to `cache/calls/<ref>/` and takes replies from `cache/replies/<callee-slug>/1.wav ...` or `reply.wav` (a `.txt` is a typed reply and skips STT); no replies ⇒ `NO_ANSWER`, honestly. A carrier (Exotel/Twilio) goes behind the three-method `Telephony` seam. `scripts/speech_demo.py --call`

<dir> runs a whole supplier call on recordings for about ₹2. From a Claude Code cloud session `api.vachana.ai` is blocked by the egress policy; run speech steps on a laptop.

MockVoice scripts: Dona Maria Homestay does not answer once, then confirms at ₹3,200 (listing said ₹2,800); Fisherman's Rest is full; escalation calls reach the member, who takes option 1.

6. Rails: what exists, what we use, what is [verify]

Pine Labs Online (`app/rails/pinelabs.py`, docs at `pinelabs.com/docs`, read via search snippets and one pasted reference page; the docs host is blocked from cloud sessions):

Product	Used for	Status
Payment links <code>POST /api/pay/v1/paymentlink</code> (<code>pre_auth</code> , <code>allowed_payment_methods</code> , <code>expire_by</code> , <code>customer</code> , <code>callback_url</code> , <code>split_info</code>)	The block: one link per payer, CARD + UPI (+ BNPL), expiring at the deadline	Request copied from the reference; response field names [verify]
Card pre-authorisation, Capture Order, Cancel Order (5-7 day hold)	Card holds	paths [verify]
UPI One-Time Mandate (₹1 lakh, 60 days, one capture, partial ok, merchant releases, customer cannot revoke in-app)	UPI_OTM	facts confirmed from the doc
UPI Reserve Pay (block once, many debits)	UPI_RESERVE	endpoints [verify]
Credit/Debit/Cardless EMI, Offer Discovery	EMI tenures in the DM; capture settles in full	[verify]
BNPL (LazyPay)	BNPL inside limit	limit [verify], MID must be enabled
Refunds <code>POST /api/pay/v1/refunds/{order_id}</code>	rollback	[verify]
Payouts (IMPS/NEFT/UPI to beneficiaries)	pool → homestay / vendor	adapter records an instruction until beneficiaries exist
Split settlements (<code>on_hold</code> , Release Settlement)	nearest existing thing to the escrow ask	not used yet
P3P (agentic payments on UPI mandates)	the ask, reframed: P3P for N payers on one order	

The ask to Pine Labs: a group order with escrow — N blocks bound to one order, shared expiry, atomic capture, settlement to suppliers only when the order is secured, one webhook. **The walls:** one capture per card hold and per OTM; no atomic N-payer capture; EMI/BNPL are checkout-time methods; card holds live 5-7 days; MDR on cards.

Gnani: see §5. The ask: normalised entities in the transcript (amounts, dates, yes/no, ordinals), streaming duplex STT/TTS, code-switch robustness.

Delhivery: distances, not parcels. `Stay.km_to_venue` is hand-written; Delhivery Maps would make the wedding/offsite ranking real.

Env: `.env.example`. Pine Labs UAT keys stay in `.env` (git-ignored). `QUORUM_VOICE=mock|gnani`, `QUORUM_PAYMENTS=mock|pinelabs`, `QUORUM_DEMO_PASSWORD` for a public deploy.

7. Code map

```
app/
  engine.py          the state machine; constants at the top; P0_CALLS; every public
method is one thing a
                    person or the world does (trigger, gather, details, vote,
member_flips, member_approves,
                    member_withdraws, member_chooses, organiser_adjusts,
organiser_decides, rerun, close,
                    disrupt, missed, member_exits, tick)
  models.py          Trip, Member, Constraint, Plan, Stay, Leg, Activity, Vote,
Authorisation, Decision,
                    Payout, CallRecord, Event; INSTRUMENTS; inr()
  messages.py        every user-visible sentence; house rules in the docstring
  lore.py            INTERESTS, HOOKS (Goa), persona_of(), hook()
  clock.py           hand-cranked time; never call datetime.now() in engine code
  scenario.py        five friends, WEDDING and LEISURE presets, REPLIES (canned gathering
answers)
  keys.py            the Gnani speech key (committed on purpose)
  main.py            FastAPI: every action is a POST; _view() is what the UI renders;
demo knobs /rails/*
  rails/base.py      VoiceRail, PaymentsRail, LogisticsRail, CaptureFailed
  rails/mock.py      MockVoice, MockPayments (ledger, pool, fail_capture_for,
approve/refund/pay_supplier),
                    MockLogistics (fares table, stays, activities, alternatives, drift
knob)
  rails/gnani.py     GnaniSpeechVoice, Telephony, FileTelephony, parsers, scripts
  rails/gnani_speech.py Vachana TTS/STT client, CallBudget
  rails/pinelabs.py  PineLabsPayments on payment links
static/index.html    the demo UI (group chat · private DMs · rails), no build step
demo.py              three transcripts → docs/demo_transcript.txt
tests/               test_engine.py (loop), test_instruments.py, test_place.py,
test_voice.py, test_budget.py
Dockerfile, render.yaml, .dockerignore
docs/                round2-answers.md, rails.md, demo_transcript.txt, this file
```

Conventions and invariants.

- Rails are the only seam. New capability goes behind an interface, mock first. The engine must pass `tests/` on mocks.
 - Every user-visible sentence lives in `app/messages.py`. Short sentences, one ask, always a default and a deadline, never a group poll.
 - Every event goes through `_say/_rail` with a priority. Calls only through `_call`.
 - A member is never debited above `Authorisation.amount` plus a top-up they approved themselves. The pool never goes negative.
 - `_retarget` is the one place a plan is re-sized for a different set of people.
 - `_fit` is the one place the budget rule lives (proposal vs post-vote).
 - Lore is picked by `_hook`, per person, never repeated (`Trip.used_hooks`).
 - Money prints via `inr()`; f-strings use `₹{inr(x)}`.
-

8. Fixtures the tests and the demo depend on

Cast (`scenario.five_friends`): Sayan (Kolkata, organiser), Aabhas (Kolkata, **pays for 2**: Aabhas and Meera), Aditi (Bengaluru → back to Mumbai), Riya (Delhi), Karan (Mumbai). Ceilings: 24k, 25k, 20k, 23k, 19k. Interests: Sayan trekking+food, Aabhas heritage+food, Aditi water sports+trekking, Riya nightlife+reels, Karan food. Presets: `WEDDING` (venue Assagao, 2–6 Oct 2026, ₹20,000, 10%), `LEISURE` (same dates). Clock starts Sat 19 Sep 2026 10:00.

Mock inventory. Flights per city (carrier, hour, hours, ₹): Kolkata 06:00 IndiGo 6,400 · 14:00 Air India 9,900 · 19:00 IndiGo 5,900 · 21:00 SpiceJet 6,100; Bengaluru 07:00 3,800 · 12:00 Akasa 3,400 · 18:00 4,600; Delhi 05:00 IndiGo 6,100 · 11:00 Vistara 8,200 · 20:00 Akasa 5,700; Mumbai 08:00 Akasa 2,900 · 13:00 3,200 · 19:00 3,900. Return legs use the destination city's table. Stays (rate per twin room per night): Fisherman's Rest 2,400 phone-only (full); Dona Maria Homestay 2,800 phone-only (call says 3,200), 1.5 km from Assagao; Zostel 1,800 hostel; Cabana by the Cove 3,800; Sea Breeze 4,600. Activities: Dudhsagar jeep 1,200 prebook (trekking), kayaking 600 prebook, heritage walk 400 prebook, others on the day. Roads to Goa: Mumbai 590 km, Bengaluru 560 km (cab ₹16/km whole car; train 22:00, 11 h, ₹1,500). Knobs: `logistics.drift[city]` multiplies live fares; `payments.fail_capture_for` bounces a debit.

Wedding numbers (Karan silent ⇒ defaults; 4 yes, Karan no): v1 six heads, stay ₹6,400/head; per head Kolkata 18,700, Aditi 12,700, Riya 18,200, Karan 12,200; Aabhas share 37,400. After the vote: five heads, stay 7,680; Sayan 19,980 (cap 22,000), Aabhas 39,960 (44,000), Aditi 13,980 (15,400), Riya 19,480 (21,500); total 93,400 against $5 \times 20,000 = 1,00,000$. Captures = payouts = 93,400 (stay 38,400 + legs 55,000); pool 0. Aabhas's 06:00 cancelled ⇒ 19:00 at 5,900 × 2 (refund 12,800; pool 1,000; late-arrival call). Riya on Delhi ×1.6 ⇒ options 9,120 / 13,120; UPI Reserve headroom 2,020 + refund 6,100 ⇒ shortfall 1,000 ⇒ top-up 1,000; on a card hold headroom is 0 ⇒ shortfall 3,020 ⇒ top-up 3,100. Karan's nudge quotes ₹12,200 a head; a flip makes six heads, total 99,200 against $6 \times 20,000$.

Leisure numbers: v1 six heads with Dudhsagar: Kolkata 19,400, Riya 19,000, Aditi 13,900, Karan 13,400. Votes 2–2–1 fail on price ⇒ v2 same stay, one night fewer: Kolkata 17,800, Aabhas 35,600, Aditi 12,300, Riya 17,400, Karan 11,800; total 94,900 against $6 \times 19,000$. Riya never blocks ⇒ five heads, stay 5,760: Sayan 18,760 (up, inside cap 19,600), total 82,300. Karan misses the 08:00 ⇒ options IndiGo 13:00 ₹3,200, Uber Outstation ₹9,400, Air India 19:00 ₹3,900; headroom 1,400 ⇒ top-up 1,800.

Wall: wedding with `fail_capture_for = {Riya}` ⇒ three debits refunded, pool 0, LAPSED, `rerun` re-issues blocks.

9. Verifying a change

```
pip install -r requirements.txt
pytest -q                      # 53 tests, ~0.5 s
python demo.py > docs/demo_transcript.txt # regenerate after any message or number
change; commit it
./run.sh                       # UI at http://localhost:8000
python scripts/speech_demo.py --dry-run # costs of the real Gnani loop; no network
needed
```

The browser click-through used during development was a Playwright script (not committed) that drove the UI on a local uvicorn: trigger → four canned replies → +24h → four yes and one no → knobs → approvals with mixed instruments → disruptions → escalation → top-up → New trip → the wall. Serialize each click on its POST response and give the page ~150 ms to re-render before asserting; launch Chromium with `executable_path="/opt/pw-browsers/chromium"` on the cloud box. Zero `pageerror/console` errors is the bar.

10. Running it for other people

`Dockerfile` (python:3.11-slim, uvicorn, `PORT`) and `render.yaml` (free web service, Docker runtime, branch `claude/nice-ramanujan-jt9qjv`, health check `/clock`, mock rails, `QUORUM_DEMO_PASSWORD` set in the dashboard). On Render: New → Blueprint → the repo → the branch. Free plan sleeps after 15 idle minutes; trips live in memory and reset on wake. Hugging Face Spaces, Fly, Railway take the same image (`PORT=7860` on Spaces). Serverless hosts are wrong for this until persistence exists. With a password set, `/clock` stays open as the health check and everything else asks for basic auth (any username).

11. Not built yet, in priority order

1. **Pine Labs UAT end to end:** keys, MID enablement (Pay by Link, pre-auth, OTM/Reserve Pay, Payouts, BNPL), fix response field names, poll captures to `SUCCESS` and raise `CaptureFailed` on `FAILED`, confirm whether pay-by-link with `pre_auth` runs UPI as OTM, confirm the EMI flow (void hold + EMI checkout, or issuer conversion), register payout beneficiaries.
 2. **Gnani on recordings, then a phone line:** run `speech_demo.py`, record homestay replies, fill the voice-failure rows in the answers doc; Exotel or Twilio behind `Telephony`; streaming over `WebSocket`.
 3. **A real channel** (Telegram in an evening; WhatsApp Cloud API with approval): inbound text → `gather` / `vote` / `details` / `member_chooses` / `member_flips`, outbound events → sends. Votes and options as DMs, never in the group.
 4. **Live fares** behind `LogisticsRail`; `Leg.quoted` vs `price` is already the drift record.
 5. **Delhivery Maps** for `km_to_venue`.
 6. **An on-trip tab** on UPI Reserve Pay: block a per-head allowance once, debit as spent, release the rest (designed, not coded).
 7. Half-built promises: the member whose own travel breaks the limit is told, not offered trains/dates; the booking-time top-up has no clock; a lost re-hold goes back to the group rather than a quick yes/no on the next stay; `member_exits` after booking informs and stops.
 8. **Persistence:** `Engine.trips` is a dict; SQLite via a small repo class.
 9. Ground transport that re-sizes with the head count, if ever wanted, goes in `_retarget`.
-

12. What not to do

- Don't build a generic AI trip planner. Research is done by chatbots already. Quorum does consensus, verification and execution.
- Don't put the organiser's card or account anywhere in the money path.
- Don't remove deadlines or defaults to make demos faster; use `clock.advance`.
- Don't add a group poll. Votes are private; the group sees a tally.
- Don't add a call the P0 test doesn't pass. No deadline-nudge calls, no calls to listed hotels.
- Don't let Quorum front or lend. A lender fronts; Quorum is settled in full.
- Don't announce personas. They shape defaults.
- Don't paste any key into chat, a commit message or a screenshot. The Gnani key is in `app/keys.py` by decision; nothing else is.

13. Document index

File	What it is
<code>README.md</code>	the product, the states table, how to run, real vs mock, the asks, deploy, layout
<code>HANDOFF.md</code>	conventions and the next-tasks list for a human or agent builder (shorter than this)
<code>docs/round2-answers.md</code>	the eight Round-2 answers being pasted into the form; \$0 is working notes with the team decisions; <code>[verify]</code> marks unconfirmed claims
<code>docs/rails.md</code>	what exists at each rail, how the prototype uses it, the ask, the wall
<code>docs/demo_transcript.txt</code>	<code>python demo.py</code> output; regenerate on change
<code>docs/handoff-for-agents.md</code>	this document

14. Glossary

Payer / member — a person in the WhatsApp group with a phone and a way to pay. **Party / heads** — the people a payer pays for, themselves included. **Per head** — the all-in cost for one of a payer's people. **Share** — per head × party; what one payer owes. **Ceiling** — a payer's stated per-head maximum; the organiser's figure by default. **Limit** — organiser's rough budget × (1 + overshoot), per head, for the proposal. **Floor / total budget** — after the vote: the lowest ceiling among the payers in, × the heads they pay for. **Cap** — share × (1 + overshoot), rounded up to ₹100: the most a payer's block can be debited. **Headroom** — what a block can still be debited without a new approval. **Block** — a UPI mandate, card hold or prepaid link for the cap; `Authorisation` in code. **Instrument** — which kind of block. **Pool** — Quorum's merchant account. **Payout** — the pool paying a supplier. **Plan / revision** — one itinerary, versioned. **In** — voted yes and not dropped. **Flip** — a no-voter joining inside the window after the tally. **Top-up** — a fresh block asked of one payer for a difference. **Hook** — one line of place lore. **P0 / P1 / P2** — a call / a text now / a text that can wait.